Microprocessor & Assembly Language

CSE 0611-3221

The Flags Register

Presented By

Saifur Rahman

Lecturer

Dept. of CSE

NBIU

References

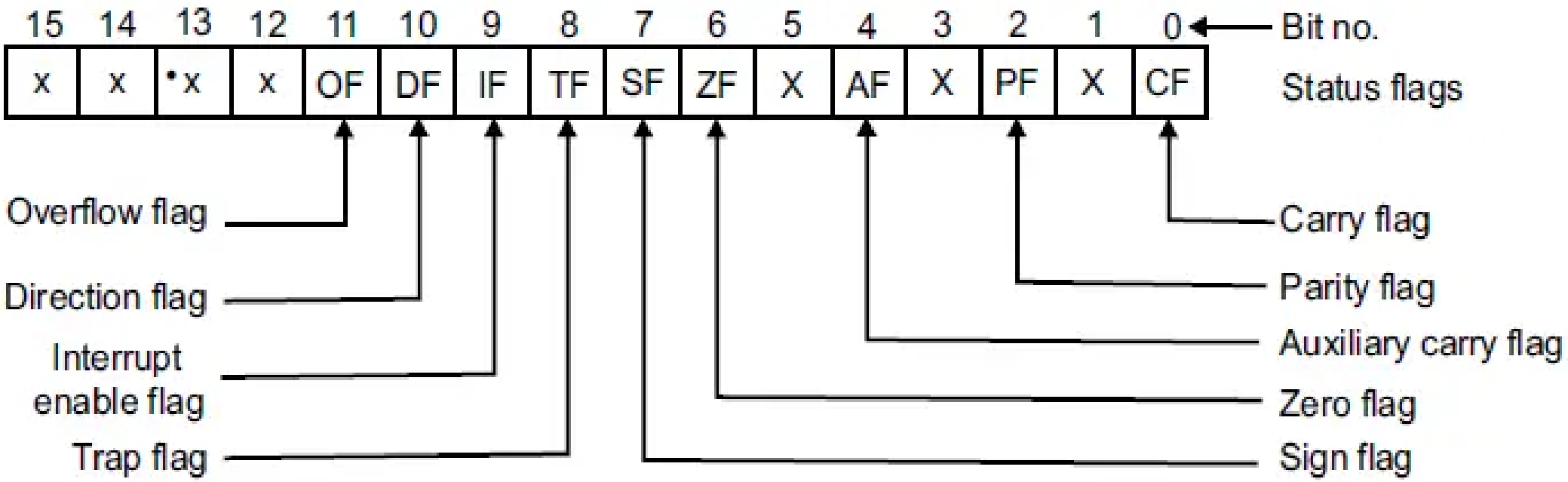
Assembly Language Programming and Organization of the IBM PC

By Ytha Yu and Charles Maru

The Flags Register

- The CPU circuits can make simple decisions based on the processor's current state. For the 8086 processor, this state is represented by **nine individual bits** called **flags**.
- **The FLAGS register** in the CPU holds flags, which are divided into:
- **Status Flags**
  - Indicate the result of a computation.
  - Located in bits 0, 2, 4, 6, 7, and 11.
- **Control Flags**
  - Enable or disable specific processor operations.
  - Located in bits 8, 9, and 10.

The Flags Register (cont'd)



The Status Flags

Carry Flag (CF)

- Set to 1 if there's a carry out from the most significant bit (MSB) during addition or a borrow into the MSB during subtraction.
- 0 if there's no carry or borrow.
- Affected by shift and rotate instructions.

Parity Flag (PF)

- Set to 1 if the low byte of the result has an even number of 1 bits (even parity).
- Set to 0 if the low byte has an odd number of 1 bits (odd parity).
- Example: If the result of an addition is FFF1h, the low byte has 7 one bits, so PF = 0.

The Status Flags (cont'd)

Auxiliary Carry Flag (AF)

- AF = 1 if there is a carry from bit 3 during addition or a borrow into bit 3 during subtraction.
- Mainly used in Binary-Coded Decimal (BCD) operations.

Zero Flag (ZF)

- ZF = 1 if the result is zero.
- ZF = 0 if the result is non-zero.

Sign Flag (SF)

- SF = 1 if the MSB of the result is 1 (indicating a negative result in signed interpretation).
- SF = 0 if the MSB is 0 (indicating a positive result).

The Status Flags (cont'd)

Overflow Flag (OF)

- OF = 1 if a signed overflow occurs (result is too large for the given number of bits).
- OF = 0 otherwise (no overflow).

Range of Numbers

Type	**Word (16 bit)**		**Byte (8 bit)**	
Range	Start	End	Start	End
Unsigned	**0**	**65535**	**0**	**255**
Signed	**-32768**	**32767**	**-128**	**127**

Overflow occurs when the result of an operation is too large for the given range.

Overflow

- Signed and unsigned overflows are separate issues.
- When adding numbers, there are four possible outcomes:

1. **No overflow:** The result is within the range.
2. **Signed overflow only:** The result is incorrect for signed numbers.
3. **Unsigned overflow only:** The result is incorrect for unsigned numbers.
4. **Both signed and unsigned overflows:** Both types are incorrect.

Overflow (cont'd)

Unsigned Overflow

```
AX: FFFFh ; 65535
BX: 0001h ; 1
ADD AX, BX
AX: 0000h
```

	1111	1111	1111	1111
+	0000	0000	0000	0001
1	0000	0000	0000	0000

- For an unsigned interpretation, the correct answer is 10000h = 65536.
- This value exceeds the range for a word operation.
- A carry is generated from the most significant bit (MSB).
- The stored answer in AX (0000h) is incorrect, indicating an unsigned overflow.

Overflow (cont'd)

Signed Overflow (Did not occur)

```
AX: FFFFh ; -1
BX: 0001h ; 1
ADD AX, BX
AX: 0000h ; 0
```

	1111	1111	1111	1111
+	0000	0000	0000	0001
1	0000	0000	0000	0000

- For an signed interpretation, The stored answer is correct:
  - FFFFh = -1
  - 0001h = 1
  - When adding: FFFFh + 0001h = -1 + 1 = 0
- Therefore, signed overflow did not occur.

Overflow (cont'd)

Signed Overflow

```
AX: 7FFFh  ; 32767
BX: 7FFFh  ; 32767
ADD AX, BX
AX: FFFEh  ; -2
```

	0111	1111	1111	1111
+	0111	1111	1111	1111
	1111	1111	1111	1110

- For a signed interpretation, The stored answer is incorrect:
  - 7FFFh = 32767
  - 7FFFh = 32767
  - When adding: 7FFEh + 7FFEh = FFFEh = -2
- Therefore, signed overflow occurred.

Overflow (cont'd)

Unsigned Overflow (Did not occur)

```
AX: 7FFFh
BX: 7FFFh
ADD AX, BX
AX: FFFEh
```

	0111	1111	1111	1111
+	0111	1111	1111	1111
	1111	1111	1111	1110

- For an unsigned interpretation, The stored answer is correct:
  - 7FFFh = 32767
  - 7FFFh = 32767
  - When adding: 7FFEh + 7FFEh = 32767 + 32767 = FFFEh
- Therefore, unsigned overflow did not occur.

Overflow (cont'd)

- **How the Processor Indicates Overflow?**

Signed Overflow	**OF = 1**
Unsigned Overflow	**CF = 1**

- **How the Processor Determines that Overflow Occurred?**

Signed Overflow	**OF = 1**	If the result has a different sign than expected.
Unsigned Overflow	**CF = 1**	If there is a carry out or borrow into the MSB.

Overflow (cont'd)

Unsigned Overflow (CF = 1)

1. Addition

- Happens when there is a carry out of the most significant bit (MSB).
- This means the correct result is larger than the largest unsigned number (FFFFh for a word, FFh for a byte).

2. Subtraction

- Occurs when there is a borrow into the MSB.
- This means the correct result is smaller than 0.

Overflow (cont'd)

Signed Overflow (OF = 1)

1. **Addition**

- When adding numbers with the **same sign**, overflow occurs if the result has the opposite sign.
- Adding 7FFFh + 7FFFh (both positive) gives FFFEh (negative), causing overflow.

2. **Subtraction**

- Subtracting numbers with **different signs** is like adding numbers with the same sign.
- A - (-B) is like A + B.
- -A - (+B) is like -A + -B.
- Overflow happens if the result has an unexpected sign.

Overflow (cont'd)

Signed Overflow (OF = 1)

1. **Addition of Numbers with Different Signs**

- Overflow is not possible.
- A + (-B) is essentially A - B.
- If A and B fit in the destination, so will A - B.

2. **Subtraction of Numbers with the Same Sign**

- Overflow is also not possible.

How Instructions Affect the Flags

- Each time the processor runs an instruction, flags are updated to show the result.
- Some instructions do not change any flags, change only specific flags, or may leave some flags undefined.

Instructions	Affects Flags
MOV/XCHG	None
ADD/SUB	All
INC/DEC	All except CF
NEG	All (CF = 1 unless result is 0, OF = 1 if word operand is 8000h or byte operand is 80h)

How Instructions Affect the Flags (cont'd)

1. ADD AX, BX, where AX contains FFFFh, BX contains FFFFh.

$$\begin{array}{rr} & \text{FFFFh} \\ + & \text{FFFFh} \\ \hline 1 & \text{FFFEh} \end{array}$$

The result stored in AX is FFFEh = 1111 1111 1111 1110.

SF = 1	Because the msb is 1.
PF = 0	Because there are 7 (odd number) of 1 bits in low byte.
ZF = 0	Because the result is non-zero.
CF = 1	Because there is a carry out of the msb on addition.
OF = 0	Because the sign of the stored result is the same as the operands.

How Instructions Affect the Flags (cont'd)

2. ADD AL, BL, where AL contains 80h, BL contains 80h.

$$\begin{array}{rr} & 80h \\ + & 80h \\ \hline 1 & 00h \end{array}$$

The result stored in AL is 00h = 0000 0000.

SF = 0	Because the msb is 0.
PF = 1	Because all the bits in the low byte are 0.
ZF = 1	Because the result is zero.
CF = 1	Because there is a carry out of the msb on addition.
OF = 1	Because the numbers being added are both negative, but the result is 0.

How Instructions Affect the Flags (cont'd)

3. SUB AX, BX, where AX contains 8000h, BX contains 0001h.

0001h	0000	0000	0000	0001
1's Comp	1111	1111	1111	1110 +1
- 0001h	1111	1111	1111	1111

	8000h	1000 0000 0000 0000
+	- 0001h	1111 1111 1111 1111
	7FFFh	1 0111 1111 1111 1111

The result stored in AX is 7FFFh = 0111 1111 1111 1111.

SF = 0	Because the msb is 0.
PF = 1	Because there are 8 (even number) one bits in the low byte.
ZF = 0	Because the result is nonzero.
CF = 0	Because a smaller unsigned number is subtracted from a larger one (No borrow).
OF = 1	Because in a signed sense, subtracting a positive from a negative one is like adding two negative. But the result is positive.

How Instructions Affect the Flags (cont'd)

4. NEG AX where AX contains 8000h.

8000h	1000	0000	0000	0000
1's Comp	0111	1111	1111	1111 +1
8000h	1000	0000	0000	0000

The result stored in AX is 8000h = 1000 0000 0000 0000.

SF = 1	Because the msb is 1.
PF = 1	Because there are 0 (even number) one bits in the low byte.
ZF = 0	Because the result is nonzero.
CF = 1	Because for NEG, CF is always 1 unless the result is 0.
OF = 1	Because when we number is negated we expect a sign change, but because 8000h is its own 2's comp., there is no sign change.

How Instructions Affect the Flags (cont'd)

5. INC AL where AL contains FFh.

$$\begin{array}{rr} & \text{FFh} \\ + & \text{01h} \\ \hline 1 & \text{00h} \end{array}$$

The result stored in AL is 00h = 0000 0000.

SF = 0	Because the msb is 0.
PF = 1	Because all the bits in the low byte are 0.
ZF = 1	Because the result is zero.
CF = 0	Because CF is unaffected by INC operation.
OF = 0	Because the numbers of different signs are being added.